CineComic-MML: Phase 4 and Phase 5 Code

Complete source: shot/frame pipeline and face detection, tracking and character IDs

Phase 4 - shots, frames, quality

Crops black bars, detects shots, samples frames, removes dark / blurry / near-duplicate frames and builds 3-10 candidate frames per dialogue segment. **Writes** data/shots/{id}.json and data/frames/{id}/shot_xxx/*.jpg.

Phase 5 - faces, tracks, character IDs

Detects faces on original-resolution frames (about 5 fps inside dialogue windows, plus Phase 4's candidate frames), embeds them with ArcFace, tracks them inside each shot, then clusters tracks into characters (CHAR_01, ...) with a cannot-link rule so two people on screen together are never merged. Low-confidence tracks become UNKNOWN. **Reads** data/shots/{id}.json and data/subtitles/{id}.json. **Writes** data/faces/{id}.json, data/embeddings/faces_{id}.npy, data/tracks/{id}.json.

Install and run

```
pip install -r requirements.txt
python -m src.video.phase4 --config configs/phase4.yaml --video clip_001
python -m src.vision.phase5 --config configs/phase5.yaml --video clip_001 --gt faces_gt.json
pytest tests/
```

Create empty __init__.py in src/, src/video/, src/vision/ and src/evaluation/ before running. Phase 5 labelled-face file format: {"frames": [{"t": 14.5, "faces": [{"box": [x0,y0,x1,y1], "character": "alice", "frontal": true}]}]}; label every visible face in each labelled frame, and use frame times that Phase 5 sampled (multiples of 1/det_fps).

Gates (suggested starting values; set your own after the first run)

Phase 4: boundary F1 >= 0.85; coverage 100%; no candidate outside its shot; black-border rate <= 2%; crop IoU >= 0.95.

Phase 5: recall >= 90% on medium/large frontal faces; 0 ID switches inside a shot; cluster purity >= 90%; no track spans a shot; no two simultaneous tracks share a character_id.

Note: InsightFace model packs are provided for non-commercial research use; check the current licence before relying on them.

Files

#	File	Purpose
1	`requirements.txt`	Python dependencies for Phases 4 and 5.
2	`configs/phase4.yaml`	All Phase 4 thresholds and settings.
3	`src/video/black_bar.py`	Black-bar (letterbox) detection; crop box in original pixels.
4	`src/video/shot_detection.py`	Shot boundary detection with PySceneDetect.
5	`src/video/quality_filter.py`	Sharpness, brightness, pHash; dark, blur and duplicate filters.
6	`src/video/frame_extraction.py`	Sampling plan and the sequential decode pass that saves cropped frames.
7	`src/evaluation/shot_metrics.py`	Boundary precision/recall/F1 and box IoU (also used by Phase 5).
8	`src/video/phase4.py`	Phase 4 stage: run, segment candidates, evaluate, gate, contact sheet, CLI.
9	`tests/test_phase4.py`	Phase 4 tests on a synthetic video.
10	`configs/phase5.yaml`	All Phase 5 thresholds and settings.
11	`src/vision/face_detection.py`	InsightFace detector + ArcFace embeddings, face quality, mouth point.
12	`src/vision/tracking.py`	Within-shot tracking (Hungarian on IoU + embedding) and track summaries.
13	`src/vision/clustering.py`	Average-linkage clustering with cannot-link constraints.
14	`src/evaluation/face_metrics.py`	Detection P/R, pairwise F1, B-cubed, purity, ID switches.
15	`src/vision/phase5.py`	Phase 5 stage: detect, track, cluster, write artifacts, evaluate, gate, CLI.
16	`tests/test_phase5.py`	Phase 5 tests with a fake detector on a synthetic video.

1. requirements.txt

Python dependencies for Phases 4 and 5.

```
opencv-python>=4.8
scenedetect[opencv]>=0.6.2
imagehash>=4.3
Pillow
numpy
pyyaml
pytest
insightface>=0.7.3
onnxruntime
scipy
```

Part A - Phase 4

2. configs/phase4.yaml

All Phase 4 thresholds and settings.

```
data_dir: data
results_dir: results
seed: 42

phase4:
  black_bar:
    n_samples: 30          # frames sampled across the video
    thresh: 16.0           # row/col mean intensity below this = black bar
    min_keep_frac: 0.5     # if crop keeps less than this area, fall back to full frame
  shots:
    detector: content      # content | adaptive
    threshold: 27.0        # ContentDetector threshold (adaptive: adaptive_threshold)
    min_scene_len_s: 0.5
  sampling:
    fps: 2.0               # regular grid, plus first/middle/last frame of each shot
    window_pad_s: 0.5      # dialogue window = [start - pad, end + pad] within its shot
    max_candidates_per_segment: 10
    min_per_segment: 3
    jpeg_quality: 90
  quality:
    min_brightness: 8.0    # rejects black frames from fades
    blur_rel_thresh: 0.4   # blurry if sharpness < this * video-median sharpness (relative, not absolute)
    phash_hamming: 5       # near-duplicate if Hamming distance <= this (within a shot)
    border_dark_thresh: 16.0
  contact_sheet: {n_shots: 20, cols: 5, thumb_w: 256}
  gate:                    # suggested starting values; justify your own in the report
    boundary_f1: 0.85
    boundary_tol_s: 0.5
    coverage: 1.0
    crop_iou: 0.95
    max_black_border_rate: 0.02
```

3. src/video/black_bar.py

Black-bar (letterbox) detection; crop box in original pixels.

```
"""Black-bar (letterbox/pillarbox) detection. Returns crop box [x0, y0, x1, y1] in ORIGINAL pixels."""
from __future__ import annotations
import cv2
import numpy as np


def crop_from_gray_frames(grays: list[np.ndarray], thresh: float, min_keep_frac: float) -> list[int]:
    H, W = grays[0].shape
    full = [0, 0, W, H]
    # A row is "bar" only if it stays dark in EVERY sampled frame (max over frames of the row mean).
    row_max = np.max([g.mean(axis=1) for g in grays], axis=0)
    rows = np.where(row_max > thresh)[0]
    if len(rows) == 0:
        return full
    y0, y1 = int(rows[0]), int(rows[-1]) + 1
    # Column profile measured only inside the content rows, so horizontal bars don't dilute it.
    col_max = np.max([g[y0:y1].mean(axis=0) for g in grays], axis=0)
    cols = np.where(col_max > thresh)[0]
    if len(cols) == 0:
        return full
    x0, x1 = int(cols[0]), int(cols[-1]) + 1
    if (x1 - x0) * (y1 - y0) < min_keep_frac * W * H:
```

```
        return full
    return [x0, y0, x1, y1]


def detect_black_bars(video_path, n_samples=30, thresh=16.0, min_keep_frac=0.5) -> list[int]:
    cap = cv2.VideoCapture(str(video_path))
    n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    W = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)); H = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    idxs = (np.linspace(0, n - 1, n_samples + 2)[1:-1] if n > n_samples else np.arange(n)).astype(int)
    grays = []
    for i in idxs:
        cap.set(cv2.CAP_PROP_POS_FRAMES, int(i))
        ok, f = cap.read()
        if ok:
            grays.append(cv2.cvtColor(f, cv2.COLOR_BGR2GRAY))
    cap.release()
    if not grays:
        return [0, 0, W, H]
    return crop_from_gray_frames(grays, thresh, min_keep_frac)
```

4. src/video/shot_detection.py

Shot boundary detection with PySceneDetect.

```
"""Shot boundary detection with PySceneDetect. Returns [{'shot_id', 'start', 'end'}] in seconds."""
from __future__ import annotations
from scenedetect import detect, ContentDetector, AdaptiveDetector


def detect_shots(video_path, duration_s: float, fps: float, detector="content",
                 threshold=27.0, min_scene_len_s=0.5) -> list[dict]:
    min_len = max(1, int(round(min_scene_len_s * fps)))
    if detector == "adaptive":
        det = AdaptiveDetector(adaptive_threshold=threshold, min_scene_len=min_len)
    else:
        det = ContentDetector(threshold=threshold, min_scene_len=min_len)
    scenes = detect(str(video_path), det)
    if not scenes:  # no cut found: the whole video is one shot
        return [{"shot_id": 0, "start": 0.0, "end": float(duration_s)}]
    shots = [{"shot_id": k, "start": s.get_seconds(), "end": e.get_seconds()}
             for k, (s, e) in enumerate(scenes)]
    shots[0]["start"] = 0.0
    shots[-1]["end"] = float(duration_s)
    return shots
```

5. src/video/quality_filter.py

Sharpness, brightness, pHash; dark, blur and duplicate filters.

```
"""Per-frame quality statistics and the relative blur / duplicate / dark filters."""
from __future__ import annotations
from collections import defaultdict
import cv2
import imagehash
import numpy as np
from PIL import Image


def frame_stats(crop_bgr: np.ndarray, border_thresh: float = 16.0) -> dict:
    gray = cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2GRAY)
    ph = imagehash.phash(Image.fromarray(cv2.cvtColor(crop_bgr, cv2.COLOR_BGR2RGB)))
    return {
        "sharp": float(cv2.Laplacian(gray, cv2.CV_64F).var()),
        "brightness": float(gray.mean()),
        "contrast": float(gray.std()),
        "phash": str(ph),
        "border_dark": has_black_bars(gray, border_thresh),
    }


def has_black_bars(gray: np.ndarray, thresh: float = 16.0) -> bool:
    return bool(gray[0].mean() < thresh or gray[-1].mean() < thresh
                or gray[:, 0].mean() < thresh or gray[:, -1].mean() < thresh)


def hamming_hex(a: str, b: str) -> int:
    return bin(int(a, 16) ^ int(b, 16)).count("1")


def apply_filters(frames: list[dict], q: dict) -> None:
```

```
    """Sets rec['reject'] to None | 'dark' | 'blurry' | 'duplicate' (in place)."""
    for f in frames:
        f["reject"] = "dark" if f["brightness"] < q["min_brightness"] else None
    live = [f for f in frames if f["reject"] is None]
    if not live:
        return
    median = float(np.median([f["sharp"] for f in live]))
    by_shot = defaultdict(list)
    for f in live:
        by_shot[f["shot_id"]].append(f)
    for fs in by_shot.values():
        sharpest = max(fs, key=lambda f: f["sharp"])  # a shot never loses its best frame to blur
        for f in fs:
            if f is not sharpest and f["sharp"] < q["blur_rel_thresh"] * median:
                f["reject"] = "blurry"
    # Near-duplicates inside a shot: sharpest of each group survives (deterministic tie-break on t)
    for fs in by_shot.values():
        kept: list[dict] = []
        for f in sorted((x for x in fs if x["reject"] is None), key=lambda x: (-x["sharp"], x["t"])):
            if any(hamming_hex(f["phash"], k["phash"]) <= q["phash_hamming"] for k in kept):
                f["reject"] = "duplicate"
            else:
                kept.append(f)
```

6. src/video/frame_extraction.py

Sampling plan and the sequential decode pass that saves cropped frames.

```
"""Sampling plan (which frame indices to decode) and one sequential decode pass."""
from __future__ import annotations
import logging
from pathlib import Path
import cv2
import numpy as np
from src.video.quality_filter import frame_stats

log = logging.getLogger("phase4")


def add_index_ranges(shots: list[dict], fps: float, n_frames: int) -> list[dict]:
    """Give every shot an inclusive frame range [i0, i1]; shots tile the video with no overlap."""
    for k, s in enumerate(shots):
        s["i0"] = min(int(round(s["start"] * fps)), n_frames - 1)
        nxt = int(round(shots[k + 1]["start"] * fps)) if k + 1 < len(shots) else n_frames
        s["i1"] = max(s["i0"], min(nxt - 1, n_frames - 1))
        s["start"], s["end"] = s["i0"] / fps, (s["i1"] + 1) / fps
    return shots


def assign_shot(seg: dict, shots: list[dict]) -> dict:
    """Shot with the largest time overlap with the segment (nearest shot if no overlap)."""
    ov = [max(0.0, min(seg["end"], s["end"]) - max(seg["start"], s["start"])) for s in shots]
    if max(ov) > 0:
        return shots[int(np.argmax(ov))]
    mid = (seg["start"] + seg["end"]) / 2
    return min(shots, key=lambda s: 0 if s["start"] <= mid < s["end"] else min(abs(mid - s["start"]), abs(
          mid - s["end"])))


def segment_window(seg: dict, shot: dict, pad: float) -> tuple[float, float]:
    lo, hi = max(shot["start"], seg["start"] - pad), min(shot["end"], seg["end"] + pad)
    if hi < lo:
        lo = hi = min(max((seg["start"] + seg["end"]) / 2, shot["start"]), shot["end"])
    return lo, hi


def build_sampling_plan(shots, segments, fps, cfg) -> dict[int, tuple[int, str]]:
    """Returns {frame_idx: (shot_id, frame_id)}. frame_ids are numbered by time inside each shot."""
    step = max(1, int(round(fps / cfg["fps"])))
    per_shot = {}
    for s in shots:
        i0, i1 = s["i0"], s["i1"]
        per_shot[s["shot_id"]] = set(range(i0, i1 + 1, step)) | {i0, (i0 + i1) // 2, i1}
    for seg in segments:
        s = assign_shot(seg, shots)
        lo_t, hi_t = segment_window(seg, s, cfg["window_pad_s"])
        lo = min(max(int(round(lo_t * fps)), s["i0"]), s["i1"])
        hi = min(max(int(round(hi_t * fps)), lo), s["i1"])
        have = [i for i in per_shot[s["shot_id"]] if lo <= i <= hi]
        if len(have) < cfg["min_per_segment"]:  # short window: add evenly spaced extra frames
```

```
            extra = np.linspace(lo, hi, cfg["min_per_segment"]).round().astype(int)
            per_shot[s["shot_id"]] |= {int(e) for e in extra}
    plan = {}
    for sid, idxs in per_shot.items():
        for n, i in enumerate(sorted(idxs)):
            plan[i] = (sid, f"s{sid:02d}_f{n:03d}")
    return plan


def extract_frames(video_path, plan, crop_box, frames_dir: Path, fps: float,
                   jpeg_quality=90, border_thresh=16.0) -> list[dict]:
    """One sequential pass (exact frames, no seeking). Saves cropped JPEGs and computes quality stats."""
    x0, y0, x1, y1 = crop_box
    cap = cv2.VideoCapture(str(video_path))
    last, idx, records = max(plan), 0, []
    while idx <= last:
        if not cap.grab():
            log.warning("decode ended early at frame %d (expected up to %d)", idx, last)
            break
        if idx in plan:
            ok, frame = cap.retrieve()
            if ok:
                sid, fid = plan[idx]
                crop = frame[y0:y1, x0:x1]
                path = frames_dir / f"shot_{sid:03d}" / f"{fid}.jpg"
                path.parent.mkdir(parents=True, exist_ok=True)
                cv2.imwrite(str(path), crop, [cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])
                rec = {"frame_id": fid, "shot_id": sid, "idx": idx, "t": idx / fps, "path": path.as_posix(
                      )}
                rec.update(frame_stats(crop, border_thresh))
                records.append(rec)
        idx += 1
    cap.release()
    return records
```

7. src/evaluation/shot_metrics.py

Boundary precision/recall/F1 and box IoU (also used by Phase 5).

```
from __future__ import annotations


def boundary_prf(pred: list[float], gt: list[float], tol: float = 0.5) -> dict:
    """One-to-one greedy matching of cut times within +/- tol seconds."""
    pred, gt = sorted(pred), sorted(gt)
    used, tp = set(), 0
    for p in pred:
        cands = [(abs(p - g), j) for j, g in enumerate(gt) if j not in used and abs(p - g) <= tol]
        if cands:
            used.add(min(cands)[1]); tp += 1
    P = tp / len(pred) if pred else (1.0 if not gt else 0.0)
    R = tp / len(gt) if gt else (1.0 if not pred else 0.0)
    F = 2 * P * R / (P + R) if P + R else 0.0
    return {"boundary_precision": P, "boundary_recall": R, "boundary_f1": F, "tp": tp}


def box_iou(a, b) -> float:
    ix = max(0, min(a[2], b[2]) - max(a[0], b[0])); iy = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ix * iy
    union = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter
    return inter / union if union else 0.0
```

8. src/video/phase4.py

Phase 4 stage: run, segment candidates, evaluate, gate, contact sheet, CLI.

```
"""Phase 4: shot detection, frame sampling, quality filtering, per-segment candidate frames.

Reads : data/raw/{id}.*, data/subtitles/{id}.json (Phase 2; optional for standalone runs)
Writes: data/shots/{id}.json (+ .metrics.json), data/frames/{id}/shot_xxx/*.jpg

Run:  python -m src.video.phase4 --config configs/phase4.yaml --video clip_001
"""
from __future__ import annotations
import argparse, hashlib, json, logging, random, time
from collections import defaultdict
from pathlib import Path

import cv2
import numpy as np
```

```
import yaml

from src.evaluation.shot_metrics import boundary_prf, box_iou
from src.video.black_bar import detect_black_bars
from src.video.frame_extraction import (add_index_ranges, assign_shot, build_sampling_plan,
                                        extract_frames, segment_window)
from src.video.quality_filter import apply_filters
from src.video.shot_detection import detect_shots

log = logging.getLogger("phase4")


def build_segment_candidates(segments, shots, frames, cfg) -> tuple[dict, int]:
    """segment_id -> [frame_id]. Guarantees min_per_segment where the shot has enough usable frames."""
    by_shot = defaultdict(list)
    for f in frames:
        by_shot[f["shot_id"]].append(f)
    mx, mn, pad = cfg["max_candidates_per_segment"], cfg["min_per_segment"], cfg["window_pad_s"]
    out, backfilled = {}, 0
    for seg in segments:
        shot = assign_shot(seg, shots)
        lo, hi = segment_window(seg, shot, pad)
        pool = by_shot[shot["shot_id"]]
        chosen = [f for f in pool if f["reject"] is None and lo <= f["t"] <= hi]
        if len(chosen) > mx:  # cap, evenly spaced in time
            sel = sorted(set(np.linspace(0, len(chosen) - 1, mx).round().astype(int)))
            chosen = [chosen[i] for i in sel]
        if len(chosen) < mn:  # backfill with the sharpest blurry/duplicate frames from the window
            rej = sorted((f for f in pool if f["reject"] in ("blurry", "duplicate") and lo <= f["t"] <= hi
                  ),
                         key=lambda f: (-f["sharp"], f["t"]))
            for f in rej[: mn - len(chosen)]:
                f["reject"], f["backfilled"] = None, True
                chosen.append(f); backfilled += 1
        if len(chosen) < mn:  # still short: nearest kept frames of the same shot
            mid = (seg["start"] + seg["end"]) / 2
            ids = {f["frame_id"] for f in chosen}
            near = sorted((f for f in pool if f["reject"] is None and f["frame_id"] not in ids),
                          key=lambda f: abs(f["t"] - mid))
            chosen += near[: mn - len(chosen)]
        out[str(seg["segment_id"])] = [f["frame_id"] for f in sorted(chosen, key=lambda f: f["t"])]
    return out, backfilled


class Phase4Stage:
    name = "shots"
    requires = ["subtitles"]

    def __init__(self, cfg: dict, video_id: str):
        self.cfg, self.p4, self.video_id = cfg, cfg["phase4"], video_id
        d = Path(cfg["data_dir"])
        found = sorted((d / "raw").glob(f"{video_id}.*"))
        if not found:
            raise FileNotFoundError(f"no video data/raw/{video_id}.*")
        self.video_path = found[0]
        self.out = d / "shots" / f"{video_id}.json"
        self.metrics_path = self.out.with_suffix(".metrics.json")
        self.frames_dir = d / "frames" / video_id
        self.transcript_path = d / "subtitles" / f"{video_id}.json"
        self.results_dir = Path(cfg.get("results_dir", "results")) / "phase4"

    # ---------- helpers ----------
    def cache_key(self) -> str:
        blob = json.dumps({"p4": self.p4, "seed": self.cfg.get("seed")}, sort_keys=True)
        return hashlib.sha1(blob.encode()).hexdigest()[:8]

    def _segments(self) -> list[dict]:
        if not self.transcript_path.exists():
            log.warning("no transcript at %s: segment_candidates will be empty", self.transcript_path)
            return []
        segs = json.loads(self.transcript_path.read_text(encoding="utf-8"))["segments"]
        return sorted(segs, key=lambda s: s["start"])

    # ---------- main work ----------
    def run(self, force=False) -> None:
        if self.out.exists() and not force:
            if json.loads(self.out.read_text()).get("config_hash") == self.cache_key():
                log.info("cache hit for %s, skipping", self.video_id); return
        random.seed(self.cfg.get("seed", 42)); np.random.seed(self.cfg.get("seed", 42))
        cap = cv2.VideoCapture(str(self.video_path))
```

```
        fps = cap.get(cv2.CAP_PROP_FPS); n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        cap.release()
        duration = n_frames / fps
        p = self.p4

        crop_box = detect_black_bars(self.video_path, **p["black_bar"])
        log.info("crop_box=%s", crop_box)
        shots = detect_shots(self.video_path, duration, fps, p["shots"]["detector"],
                             p["shots"]["threshold"], p["shots"]["min_scene_len_s"])
        add_index_ranges(shots, fps, n_frames)
        log.info("%d shots", len(shots))

        segments = self._segments()
        plan = build_sampling_plan(shots, segments, fps, p["sampling"])
        log.info("decoding %d sampled frames", len(plan))
        if self.frames_dir.exists():  # stale frames from an earlier config must not survive
            for old in self.frames_dir.rglob("*.jpg"):
                old.unlink()
        frames = extract_frames(self.video_path, plan, crop_box, self.frames_dir, fps,
                                p["sampling"]["jpeg_quality"], p["quality"]["border_dark_thresh"])
        apply_filters(frames, p["quality"])
        cands, backfilled = build_segment_candidates(segments, shots, frames, p["sampling"])

        n_removed = defaultdict(int)
        for f in frames:
            if f["reject"] is not None:
                n_removed[f["reject"]] += 1
                Path(f["path"]).unlink(missing_ok=True)
        kept = [f for f in frames if f["reject"] is None]
        by_shot = defaultdict(list)
        for f in sorted(kept, key=lambda f: f["t"]):
            rec = {k: (round(f[k], 3) if isinstance(f[k], float) else f[k])
                   for k in ("frame_id", "idx", "t", "sharp", "brightness", "phash", "path", "border_dark"
                         )}
            if f.get("backfilled"):
                rec["backfilled"] = True
            by_shot[f["shot_id"]].append(rec)

        doc = {
            "video_id": self.video_id, "fps": fps, "n_frames": n_frames, "duration_s": duration,
            "crop_box": crop_box, "config_hash": self.cache_key(),
            "shots": [{"shot_id": s["shot_id"], "start": round(s["start"], 4), "end": round(s["end"], 4),
                       "frames": by_shot[s["shot_id"]]} for s in shots],
            "segment_candidates": cands,
            "stats": {"sampled": len(frames), "kept": len(kept), "removed": dict(n_removed),
                      "backfilled": backfilled, "n_segments": len(segments)},
        }
        self.out.parent.mkdir(parents=True, exist_ok=True)
        self.out.write_text(json.dumps(doc, indent=1), encoding="utf-8")
        self.make_contact_sheet(doc)

    # ---------- evaluation ----------
    def evaluate(self, gt: dict | None = None) -> dict:
        doc = json.loads(self.out.read_text())
        g = self.p4["gate"]; mn = self.p4["sampling"]["min_per_segment"]
        frames = {f["frame_id"]: (s, f) for s in doc["shots"] for f in s["frames"]}
        kept = [f for s in doc["shots"] for f in s["frames"]]
        cands = doc["segment_candidates"]
        m = {
            "n_shots": len(doc["shots"]), "n_frames_kept": len(kept), **{f"stats_{k}": v for k, v in doc["
                  stats"].items() if k != "removed"},
            "removed": doc["stats"]["removed"],
            "pct_removed": 1 - doc["stats"]["kept"] / max(1, doc["stats"]["sampled"]),
            "pool_per_min": len(kept) / (doc["duration_s"] / 60),
            "black_border_rate": sum(f["border_dark"] for f in kept) / max(1, len(kept)),
        }
        if cands:
            m["coverage"] = sum(len(v) >= mn for v in cands.values()) / len(cands)
            # unit-test style invariant: every candidate lies inside the shot it belongs to
            bad = 0
            for ids in cands.values():
                for fid in ids:
                    s, f = frames[fid]
                    bad += not (s["start"] - 1e-6 <= f["t"] <= s["end"] + 1e-6)
            m["candidates_outside_shot"] = bad
        if gt:
            if "boundaries" in gt:
                pred = [s["start"] for s in doc["shots"][1:]]
                m.update(boundary_prf(pred, gt["boundaries"], g["boundary_tol_s"]))
            if "crop_box" in gt:
```

```
                m["crop_iou"] = box_iou(doc["crop_box"], gt["crop_box"])
        return m

    def gate(self, m: dict) -> tuple[bool, str]:
        g, fails = self.p4["gate"], []
        if "coverage" in m and m["coverage"] < g["coverage"]: fails.append(f"coverage {m['coverage']:.2f}"
            )
        if m.get("candidates_outside_shot", 0): fails.append("candidates outside their shot")
        if m["black_border_rate"] > g["max_black_border_rate"]: fails.append(f"black borders {m['black_bor
            der_rate']:.3f}")
        if "boundary_f1" in m and m["boundary_f1"] < g["boundary_f1"]: fails.append(f"boundary F1 {m['boun
            dary_f1']:.2f}")
        if "crop_iou" in m and m["crop_iou"] < g["crop_iou"]: fails.append(f"crop IoU {m['crop_iou']:.2f}"
            )
        return (not fails, "; ".join(fails) or "all gates passed")

    def execute(self, gt=None, force=False) -> bool:
        t0 = time.time()
        self.run(force=force)
        m = self.evaluate(gt)
        ok, why = self.gate(m)
        m.update(runtime_s=time.time() - t0, passed=ok, reason=why, cfg=self.cache_key())
        self.metrics_path.write_text(json.dumps(m, indent=2))
        log.info("metrics: %s", json.dumps(m))
        return ok

    # ---------- manual visual check ----------
    def make_contact_sheet(self, doc: dict) -> None:
        cs = self.p4["contact_sheet"]
        shots = [s for s in doc["shots"] if s["frames"]]
        if not shots:
            return
        rnd = random.Random(self.cfg.get("seed", 42))
        pick = sorted(rnd.sample(shots, min(cs["n_shots"], len(shots))), key=lambda s: s["shot_id"])
        tw, tiles = cs["thumb_w"], []
        for s in pick:
            img = cv2.imread(s["frames"][len(s["frames"]) // 2]["path"])
            th = int(img.shape[0] * tw / img.shape[1])
            img = cv2.resize(img, (tw, th))
            cv2.putText(img, f"shot {s['shot_id']}", (6, 18), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255)
                , 2)
            tiles.append(cv2.resize(img, (tw, int(tw * 9 / 16))))
        while len(tiles) % cs["cols"]:
            tiles.append(np.zeros_like(tiles[0]))
        rows = [np.hstack(tiles[i:i + cs["cols"]]) for i in range(0, len(tiles), cs["cols"])]
        self.results_dir.mkdir(parents=True, exist_ok=True)
        cv2.imwrite(str(self.results_dir / f"{self.video_id}_contact.jpg"), np.vstack(rows))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default="configs/phase4.yaml")
    ap.add_argument("--video", required=True)
    ap.add_argument("--force", action="store_true")
    ap.add_argument("--gt-boundaries", help="JSON list of manual cut times in seconds")
    ap.add_argument("--gt-crop", help="manual crop box 'x0,y0,x1,y1'")
    a = ap.parse_args()
    cfg = yaml.safe_load(open(a.config))
    Path("logs").mkdir(exist_ok=True)
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
                        handlers=[logging.FileHandler(f"logs/{a.video}.log"), logging.StreamHandler()])
    gt = {}
    if a.gt_boundaries: gt["boundaries"] = json.load(open(a.gt_boundaries))
    if a.gt_crop: gt["crop_box"] = [int(x) for x in a.gt_crop.split(",")]
    ok = Phase4Stage(cfg, a.video).execute(gt or None, force=a.force)
    raise SystemExit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

9. tests/test_phase4.py

Phase 4 tests on a synthetic video.

```
import json
from pathlib import Path
import cv2
import numpy as np
import pytest
```

```
import yaml

pytest.importorskip("scenedetect")
pytest.importorskip("imagehash")
from src.evaluation.shot_metrics import boundary_prf, box_iou
from src.video.black_bar import crop_from_gray_frames
from src.video.frame_extraction import add_index_ranges, assign_shot
from src.video.phase4 import Phase4Stage
from src.video.quality_filter import apply_filters, hamming_hex

FPS, W, H, BAR = 25, 320, 240, 20


def _make_video(path: Path, shots=3, sec=2):
    vw = cv2.VideoWriter(str(path), cv2.VideoWriter_fourcc(*"mp4v"), FPS, (W, H))
    for k in range(shots):
        rs = np.random.RandomState(k)
        tex = cv2.resize(rs.randint(40, 255, (24, 40, 3), dtype=np.uint8), (W, H - 2 * BAR), interpolation
              =cv2.INTER_NEAREST)
        for i in range(FPS * sec):
            f = np.zeros((H, W, 3), np.uint8)
            f[BAR:H - BAR] = tex
            cv2.rectangle(f, (10 + i, BAR + 10), (40 + i, BAR + 40), (255, 255, 255), -1)  # slight motion
            vw.write(f)
    vw.release()


@pytest.fixture
def stage(tmp_path):
    cfg = yaml.safe_load(open(Path(__file__).parents[1] / "configs" / "phase4.yaml"))
    cfg["data_dir"], cfg["results_dir"] = str(tmp_path / "data"), str(tmp_path / "results")
    (tmp_path / "data" / "raw").mkdir(parents=True); (tmp_path / "data" / "subtitles").mkdir(parents=True)
    _make_video(tmp_path / "data" / "raw" / "t.mp4")
    segs = [{"segment_id": 1, "start": 0.2, "end": 1.0}, {"segment_id": 2, "start": 2.3, "end": 3.1},
            {"segment_id": 3, "start": 4.5, "end": 5.5}]
    (tmp_path / "data" / "subtitles" / "t.json").write_text(json.dumps({"segments": segs}))
    return Phase4Stage(cfg, "t")


def test_black_bar_crop():
    g = np.zeros((H, W), np.uint8); g[BAR:H - BAR] = 120
    assert crop_from_gray_frames([g, g], 16.0, 0.5) == [0, BAR, W, H - BAR]


def test_boundary_and_iou():
    assert boundary_prf([2.0, 4.1], [2.0, 4.0, 9.0], 0.5)["tp"] == 2
    assert box_iou([0, 0, 10, 10], [0, 0, 10, 10]) == 1.0


def test_dark_and_duplicate_filters():
    q = {"min_brightness": 8, "blur_rel_thresh": 0.4, "phash_hamming": 5}
    mk = lambda i, sharp, br, ph: dict(frame_id=i, shot_id=0, t=float(i), sharp=sharp, brightness=br, phas
          h=ph)
    fr = [mk(0, 100, 2, "0" * 16), mk(1, 100, 90, "0" * 16), mk(2, 120, 90, "0" * 15 + "1"), mk(3, 100, 90
          , "f" * 16)]
    apply_filters(fr, q)
    assert fr[0]["reject"] == "dark" and fr[1]["reject"] == "duplicate" and fr[2]["reject"] is None and fr
          [3]["reject"] is None
    assert hamming_hex("0" * 16, "f" * 16) == 64


def test_pipeline_end_to_end_and_determinism(stage):
    assert stage.execute(gt={"boundaries": [2.0, 4.0], "crop_box": [0, BAR, W, H - BAR]}, force=True)
    doc1 = json.loads(stage.out.read_text())
    m = json.loads(stage.metrics_path.read_text())
    assert doc1["crop_box"] == [0, BAR, W, H - BAR]
    assert len(doc1["shots"]) == 3 and m["boundary_f1"] == 1.0
    assert m["coverage"] == 1.0 and m["candidates_outside_shot"] == 0 and m["black_border_rate"] == 0
    shot_of = {f["frame_id"]: s for s in doc1["shots"] for f in s["frames"]}
    for sid, ids in doc1["segment_candidates"].items():  # all candidates inside the segment's shot
        assert len({shot_of[i]["shot_id"] for i in ids}) == 1
    for s in doc1["shots"]:  # saved frames have no bars
        img = cv2.imread(s["frames"][0]["path"]); assert img.shape[:2] == (H - 2 * BAR, W)
    stage.run(force=True)  # same input + config => same frame list
    doc2 = json.loads(stage.out.read_text())
    assert [f["frame_id"] for s in doc1["shots"] for f in s["frames"]] == [f["frame_id"] for s in doc2["sh
          ots"] for f in s["frames"]]
```

Part B - Phase 5

10. configs/phase5.yaml

All Phase 5 thresholds and settings.

```
data_dir: data
results_dir: results
seed: 42

phase5:
  detection:
    model_name: buffalo_l      # InsightFace pack: SCRFD detector + ArcFace (512-d). CHECK its licence (re
          search use).
    model_root: null           # null = ~/.insightface
    det_size: [640, 640]       # raise (e.g. 960) if small faces are missed, at a speed cost
    use_gpu: false
    det_fps: 5                 # detection rate inside dialogue windows
    window_pad_s: 0.5
    min_face_px: 40            # discard faces smaller than this (min of width/height)
    min_det_score: 0.5
    ref_px: 112                # face height at which the size term of quality saturates
  tracking:
    max_gap_s: 1.0             # a track survives this long without a match
    iou_weight: 0.5            # score = w*IoU + (1-w)*cosine(embedding)
    iou_min: 0.1               # a pair is eligible if IoU >= iou_min OR cosine >= emb_gate
    emb_gate: 0.5
    emb_veto: 0.1              # cosine below this blocks a match even if the boxes overlap
    min_score: 0.3
    ema: 0.3                   # running track-embedding update weight
  identity:
    top_k: 5                   # track embedding = mean of its top-k quality faces
    cluster_threshold: 0.55    # cosine DISTANCE for merging tracks (tune on labelled faces)
    min_track_faces: 2
    min_track_quality: 0.25
    min_id_conf: 0.40          # below this the track is UNKNOWN instead of forced into a cluster
  track_sheet: {cols: 8, thumb: 112}
  eval:
    tol_s: 0.06                # GT frame time vs detection time tolerance
    iou_thr: 0.5
    size_bins: [80, 160]       # face height px: small < 80 <= medium < 160 <= large
  gate:                        # suggested starting values; justify your own in the report
    medium_large_frontal_recall: 0.90
    max_id_switches: 0
    cluster_purity: 0.90
```

11. src/vision/face_detection.py

InsightFace detector + ArcFace embeddings, face quality, mouth point.

```
"""Face detection + ArcFace embeddings (InsightFace) and per-face quality / mouth point.

All coordinates are ORIGINAL-FRAME pixels (the full decoded frame, not the black-bar-cropped one).
"""
from __future__ import annotations
import numpy as np


class InsightFaceAnalyzer:
    """SCRFD detector + ArcFace recogniser from the InsightFace 'buffalo_l' pack.
    analyze(bgr) -> [{'box': [x0,y0,x1,y1], 'kps': 5x2, 'det': float, 'emb': 512-d L2-normalised}]
    Any object with this analyze() method can replace it (the tests use a fake one)."""

    def __init__(self, model_name="buffalo_l", det_size=(640, 640), use_gpu=False, model_root=None):
        from insightface.app import FaceAnalysis  # imported lazily so the rest of the code needs no GPU s
              tack
        providers = ["CUDAExecutionProvider", "CPUExecutionProvider"] if use_gpu else ["CPUExecutionProvid
              er"]
        kw = {"root": model_root} if model_root else {}
        self.app = FaceAnalysis(name=model_name, providers=providers,
                                allowed_modules=["detection", "recognition"], **kw)
        self.app.prepare(ctx_id=0 if use_gpu else -1, det_size=tuple(det_size))

    def analyze(self, bgr: np.ndarray) -> list[dict]:
        return [{"box": f.bbox.tolist(), "kps": f.kps.tolist(), "det": float(f.det_score),
                 "emb": f.normed_embedding.astype(np.float32)} for f in self.app.get(bgr)]


def face_quality(box, kps, det_score: float, ref_px: float = 112.0) -> tuple[float, float]:
```

```
    """Returns (quality in [0,1], frontality in [0,1]).
    quality = detector confidence * size term * (0.5 + 0.5 * frontality).
    Frontality is a cheap yaw proxy: nose offset from the eye midpoint, relative to the eye distance."""
    w, h = box[2] - box[0], box[3] - box[1]
    size = min(1.0, min(w, h) / ref_px)
    eye_mid = (kps[0][0] + kps[1][0]) / 2
    eye_d = abs(kps[1][0] - kps[0][0]) + 1e-6
    frontal = 1.0 - min(1.0, abs(kps[2][0] - eye_mid) / eye_d)
    return float(det_score * size * (0.5 + 0.5 * frontal)), float(frontal)


def mouth_centre(kps) -> list[float]:
    """InsightFace 5-point order: L-eye, R-eye, nose, L-mouth-corner, R-mouth-corner."""
    return [float((kps[3][0] + kps[4][0]) / 2), float((kps[3][1] + kps[4][1]) / 2)]


def detect_faces(analyzer, frame: np.ndarray, cfg: dict) -> list[dict]:
    """Detect, filter by size / confidence, add quality + mouth. Each result carries 'emb' (np array)."""
    H, W = frame.shape[:2]
    out = []
    for d in analyzer.analyze(frame):
        x0, y0, x1, y1 = d["box"]
        x0, y0, x1, y1 = max(0, x0), max(0, y0), min(W, x1), min(H, y1)
        if min(x1 - x0, y1 - y0) < cfg["min_face_px"] or d["det"] < cfg["min_det_score"]:
            continue
        q, frontal = face_quality([x0, y0, x1, y1], d["kps"], d["det"], cfg["ref_px"])
        out.append({"box": [int(round(x0)), int(round(y0)), int(round(x1)), int(round(y1))],
                    "kps": [[round(float(x), 1), round(float(y), 1)] for x, y in d["kps"]],
                    "mouth": [round(v, 1) for v in mouth_centre(d["kps"])],
                    "det": round(d["det"], 4), "q": round(q, 4), "frontal": round(frontal, 3),
                    "emb": d["emb"]})
    return out
```

12. src/vision/tracking.py

Within-shot tracking (Hungarian on IoU + embedding) and track summaries.

```
"""Within-shot face tracking: Hungarian matching on a mix of box IoU and embedding cosine similarity."""
from __future__ import annotations
import numpy as np
from scipy.optimize import linear_sum_assignment
from src.evaluation.shot_metrics import box_iou


def _unit(v):
    return v / (np.linalg.norm(v) + 1e-12)


def track_shot(faces: list[dict], embs: np.ndarray, cfg: dict, start_id: int = 0):
    """faces: dicts with face_idx, t, box (all from ONE shot). embs[face_idx] = unit ArcFace vector.
    Returns (tracks, next_track_id); sets face['track_id']. A track never leaves the shot it was built in,
    and one face is never assigned to two tracks (one-to-one matching), so two faces in the same frame
    always get different tracks."""
    by_t: dict[float, list[dict]] = {}
    for f in faces:
        by_t.setdefault(f["t"], []).append(f)
    tracks, active, nid = [], [], start_id
    for t in sorted(by_t):
        dets = by_t[t]
        active = [tr for tr in active if t - tr["last_t"] <= cfg["max_gap_s"]]
        matched = set()
        if active:
            S = np.zeros((len(active), len(dets))); ok = np.zeros(S.shape, bool)
            for i, tr in enumerate(active):
                for j, d in enumerate(dets):
                    iou = box_iou(tr["last_box"], d["box"])
                    cos = float(tr["emb"] @ embs[d["face_idx"]])
                    S[i, j] = cfg["iou_weight"] * iou + (1 - cfg["iou_weight"]) * cos
                    ok[i, j] = ((iou >= cfg["iou_min"] or cos >= cfg["emb_gate"])
                                and cos >= cfg["emb_veto"] and S[i, j] >= cfg["min_score"])
            rows, cols = linear_sum_assignment(np.where(ok, -S, 1e6))
            for i, j in zip(rows, cols):
                if ok[i, j]:
                    tr, d = active[i], dets[j]
                    tr["faces"].append(d); d["track_id"] = tr["track_id"]
                    tr["last_box"], tr["last_t"] = d["box"], t
                    tr["emb"] = _unit((1 - cfg["ema"]) * tr["emb"] + cfg["ema"] * embs[d["face_idx"]])
                    matched.add(j)
        for j, d in enumerate(dets):
            if j not in matched:
```

```
                tr = {"track_id": nid, "faces": [d], "last_box": d["box"], "last_t": t,
                      "emb": embs[d["face_idx"]].copy()}
                d["track_id"] = nid; nid += 1
                tracks.append(tr); active.append(tr)
    return tracks, nid


def summarise_track(tr: dict, embs: np.ndarray, top_k: int) -> tuple[np.ndarray, float]:
    """Track embedding = quality-weighted mean of its top-k faces; track quality = mean quality of those f
          aces."""
    best = sorted(tr["faces"], key=lambda f: -f["q"])[:top_k]
    w = np.array([f["q"] for f in best]) + 1e-6
    e = (embs[[f["face_idx"] for f in best]] * w[:, None]).sum(0)
    return _unit(e), float(np.mean([f["q"] for f in best]))
```

13. src/vision/clustering.py

Average-linkage clustering with cannot-link constraints.

```
"""Average-linkage agglomerative clustering (cosine distance) with cannot-link constraints.

Plain sklearn clustering cannot forbid two tracks that are on screen at the same time from being merged
into one character; this small implementation can. Lance-Williams update: a blocked pair has distance inf,
and inf survives averaging, so a merged cluster stays blocked from everything either part was blocked from
      ."""
from __future__ import annotations
import numpy as np


def constrained_average_linkage(E: np.ndarray, cannot_link, threshold: float) -> np.ndarray:
    """E: (n, d) unit vectors. cannot_link: iterable of (i, j). Returns labels 0..K-1 ordered by first mem
          ber."""
    n = len(E)
    if n == 0:
        return np.zeros(0, dtype=int)
    D = 1.0 - E @ E.T
    np.fill_diagonal(D, np.inf)
    for i, j in cannot_link:
        D[i, j] = D[j, i] = np.inf
    size, labels = np.ones(n), np.arange(n)
    while n > 1:
        a, b = divmod(int(np.argmin(D)), n)
        if not D[a, b] <= threshold:      # also stops when everything left is blocked (inf)
            break
        new = (size[a] * D[a] + size[b] * D[b]) / (size[a] + size[b])
        D[a, :] = new; D[:, a] = new; D[a, a] = np.inf
        D[b, :] = np.inf; D[:, b] = np.inf
        size[a] += size[b]; labels[labels == b] = a
    remap, out = {}, np.zeros(n, dtype=int)
    for i, l in enumerate(labels):
        out[i] = remap.setdefault(l, len(remap))
    return out
```

14. src/evaluation/face_metrics.py

Detection P/R, pairwise F1, B-cubed, purity, ID switches.

```
"""Phase 5 metrics: detection P/R by size and pose, identity clustering (pairwise F1, B-cubed, purity),
ID switches and track fragmentation. Pure functions, easy to unit-test."""
from __future__ import annotations
from bisect import bisect_left, bisect_right
from collections import Counter, defaultdict
from src.evaluation.shot_metrics import box_iou


def match_boxes(gt_boxes, det_boxes, thr=0.5) -> list[tuple[int, int]]:
    """Greedy one-to-one matching by descending IoU."""
    cand = sorted(((box_iou(g, d), gi, di) for gi, g in enumerate(gt_boxes) for di, d in enumerate(det_box
          es)),
                  reverse=True)
    ug, ud, out = set(), set(), []
    for iou, gi, di in cand:
        if iou < thr: break
        if gi not in ug and di not in ud:
            ug.add(gi); ud.add(di); out.append((gi, di))
    return out


def _comb2(n): return n * (n - 1) // 2
def pairwise_f1(true, pred) -> dict:
```

```
    ct, cp, cj = Counter(true), Counter(pred), Counter(zip(true, pred))
    tp = sum(_comb2(v) for v in cj.values())
    pp, tt = sum(_comb2(v) for v in cp.values()), sum(_comb2(v) for v in ct.values())
    P, R = (tp / pp if pp else 1.0), (tp / tt if tt else 1.0)
    return {"pairwise_precision": P, "pairwise_recall": R, "pairwise_f1": 2 * P * R / (P + R) if P + R els
          e 0.0}


def purity(true, pred) -> float:
    by = defaultdict(Counter)
    for t, p in zip(true, pred): by[p][t] += 1
    return sum(max(c.values()) for c in by.values()) / len(true) if true else 1.0


def bcubed(true, pred) -> dict:
    ct, cp, cj = Counter(true), Counter(pred), Counter(zip(true, pred))
    n = len(true)
    if not n: return {"bcubed_precision": 1.0, "bcubed_recall": 1.0, "bcubed_f1": 1.0}
    P = sum(cj[(t, p)] / cp[p] for t, p in zip(true, pred)) / n
    R = sum(cj[(t, p)] / ct[t] for t, p in zip(true, pred)) / n
    return {"bcubed_precision": P, "bcubed_recall": R, "bcubed_f1": 2 * P * R / (P + R) if P + R else 0.0}


def id_switches(records: list[dict]) -> tuple[int, int]:
    """records: {shot_id, gt, track_id, t}. Per (shot, true person), ordered by time:
    switches = changes of track_id between consecutive matched faces; fragmentation = distinct tracks - 1.
          """
    groups = defaultdict(list)
    for r in records: groups[(r["shot_id"], r["gt"])].append(r)
    sw = fr = 0
    for rs in groups.values():
        ids = [r["track_id"] for r in sorted(rs, key=lambda r: r["t"])]
        sw += sum(a != b for a, b in zip(ids, ids[1:]))
        fr += len(set(ids)) - 1
    return sw, fr


def evaluate_against_gt(gt: dict, faces: list[dict], track_char: dict, cfg: dict) -> dict:
    """gt = {'frames': [{'t': s, 'faces': [{'box': [x0,y0,x1,y1], 'character': str?, 'frontal': bool?}]}]}
    Every visible face in a labelled frame must be labelled, otherwise precision is meaningless."""
    faces = sorted(faces, key=lambda f: f["t"]); ts = [f["t"] for f in faces]
    lo_b, hi_b = cfg["size_bins"]
    size_of = lambda h: "small" if h < lo_b else ("medium" if h < hi_b else "large")
    tp = fp = fn = 0
    rec = defaultdict(lambda: [0, 0])          # category -> [found, total]
    pairs, sw_recs, n_unknown = [], [], 0
    for fr in gt["frames"]:
        dets = faces[bisect_left(ts, fr["t"] - cfg["tol_s"]): bisect_right(ts, fr["t"] + cfg["tol_s"])]
        gts = fr["faces"]
        m = match_boxes([g["box"] for g in gts], [d["box"] for d in dets], cfg["iou_thr"])
        found = {gi: di for gi, di in m}
        tp += len(m); fn += len(gts) - len(m); fp += len(dets) - len(m)
        for gi, g in enumerate(gts):
            cats = [size_of(g["box"][3] - g["box"][1])]
            if "frontal" in g: cats.append("frontal" if g["frontal"] else "profile")
            if "frontal" in g and g["frontal"] and cats[0] in ("medium", "large"):
                cats.append("medium_large_frontal")
            for c in cats:
                rec[c][1] += 1; rec[c][0] += gi in found
            if gi in found and g.get("character"):
                d = dets[found[gi]]; pc = track_char.get(d["track_id"], "UNKNOWN")
                if pc == "UNKNOWN": n_unknown += 1; continue
                pairs.append((g["character"], pc))
                sw_recs.append({"shot_id": d["shot_id"], "gt": g["character"], "track_id": d["track_id"],
                      "t": d["t"]})
    P, R = tp / (tp + fp) if tp + fp else 0.0, tp / (tp + fn) if tp + fn else 0.0
    m = {"det_precision": P, "det_recall": R, "det_f1": 2 * P * R / (P + R) if P + R else 0.0,
         **{f"recall_{k}": v[0] / v[1] for k, v in rec.items() if v[1]},
         "n_gt_faces": tp + fn, "gt_unknown_among_matched": n_unknown}
    if pairs:
        t, p = [a for a, _ in pairs], [b for _, b in pairs]
        m.update(pairwise_f1(t, p)); m.update(bcubed(t, p)); m["cluster_purity"] = purity(t, p)
        m["id_switches"], m["track_fragmentation"] = id_switches(sw_recs)
    return m
```

15. src/vision/phase5.py

Phase 5 stage: detect, track, cluster, write artifacts, evaluate, gate, CLI.

```
"""Phase 5: face detection, within-shot tracking, ArcFace character IDs.
```

```
Reads : data/raw/{id}.*, data/shots/{id}.json (Phase 4), data/subtitles/{id}.json (Phase 2, optional)
Writes: data/faces/{id}.json        every detected face (box, 5 landmarks, mouth, quality, track_id, emb_i
      dx)
        data/embeddings/faces_{id}.npy   (N, 512) float32; row = face_idx = emb_idx
        data/tracks/{id}.json       tracks with character_id / id_conf (+ .metrics.json)

Run:  python -m src.vision.phase5 --config configs/phase5.yaml --video clip_001 [--gt faces_gt.json]
"""
from __future__ import annotations
import argparse, hashlib, json, logging, random, time
from bisect import bisect_right
from collections import defaultdict
from itertools import combinations
from pathlib import Path

import cv2
import numpy as np
import yaml

from src.evaluation.face_metrics import evaluate_against_gt
from src.vision.clustering import constrained_average_linkage
from src.vision.face_detection import InsightFaceAnalyzer, detect_faces
from src.vision.tracking import summarise_track, track_shot

log = logging.getLogger("phase5")
UNKNOWN = "UNKNOWN"


def build_detection_indices(fps, n_frames, segments, extra_idx, cfg) -> list[int]:
    """Frame indices to analyse: a det_fps grid inside every dialogue window (grid-aligned so overlapping
    windows share frames), plus Phase 4's candidate frames. With no transcript: the grid over the whole vi
          deo."""
    step = max(1, int(round(fps / cfg["det_fps"])))
    if not segments:
        idx = set(range(0, n_frames, step))
    else:
        idx = set()
        for s in segments:
            lo = max(0, int(round((s["start"] - cfg["window_pad_s"]) * fps)))
            hi = min(n_frames - 1, int(round((s["end"] + cfg["window_pad_s"]) * fps)))
            idx |= set(range(-(-lo // step) * step, hi + 1, step))
    idx |= {i for i in extra_idx if 0 <= i < n_frames}
    return sorted(idx)


def iter_frames(video_path, idxs):
    """Yield (frame_idx, full original-resolution BGR frame) with one sequential decode (exact frames)."""
    want = set(idxs)
    if not want:
        return
    last, i = max(want), 0
    cap = cv2.VideoCapture(str(video_path))
    while i <= last and cap.grab():
        if i in want:
            ok, fr = cap.retrieve()
            if ok:
                yield i, fr
        i += 1
    cap.release()


def check_invariants(tracks: list[dict], shots: list[dict], eps=1e-6) -> dict:
    bounds = {s["shot_id"]: (s["start"], s["end"]) for s in shots}
    span = sum(any(not (bounds[tr["shot_id"]][0] - eps <= f["t"] <= bounds[tr["shot_id"]][1] + eps)
                   for f in tr["frames"]) for tr in tracks)
    known = [tr for tr in tracks if tr["character_id"] != UNKNOWN]
    same = sum(a["character_id"] == b["character_id"] and a["t_start"] <= b["t_end"] and b["t_start"] <= a
          ["t_end"]
               for a, b in combinations(known, 2))
    return {"tracks_spanning_shots": span, "same_time_same_character": same}


class Phase5Stage:
    name = "tracks"
    requires = ["shots", "subtitles"]

    def __init__(self, cfg: dict, video_id: str, analyzer=None):
        self.cfg, self.p5, self.video_id, self._analyzer = cfg, cfg["phase5"], video_id, analyzer
        d = Path(cfg["data_dir"])
        found = sorted((d / "raw").glob(f"{video_id}.*"))
```

```
        if not found:
            raise FileNotFoundError(f"no video data/raw/{video_id}.*")
        self.video_path = found[0]
        self.shots_path = d / "shots" / f"{video_id}.json"
        self.transcript_path = d / "subtitles" / f"{video_id}.json"
        self.faces_path = d / "faces" / f"{video_id}.json"
        self.emb_path = d / "embeddings" / f"faces_{video_id}.npy"
        self.out = d / "tracks" / f"{video_id}.json"
        self.metrics_path = self.out.with_suffix(".metrics.json")
        self.results_dir = Path(cfg.get("results_dir", "results")) / "phase5"

    @property
    def analyzer(self):
        if self._analyzer is None:
            d = self.p5["detection"]
            self._analyzer = InsightFaceAnalyzer(d["model_name"], d["det_size"], d["use_gpu"], d["model_ro
                  ot"])
        return self._analyzer

    def cache_key(self) -> str:
        upstream = json.loads(self.shots_path.read_text()).get("config_hash", "") if self.shots_path.exist
              s() else ""
        blob = json.dumps({"p5": self.p5, "seed": self.cfg.get("seed"), "up": upstream}, sort_keys=True)
        return hashlib.sha1(blob.encode()).hexdigest()[:8]

    def _segments(self) -> list[dict]:
        if not self.transcript_path.exists():
            log.warning("no transcript: analysing the whole video at %s fps", self.p5["detection"]["det_fp
                  s"])
            return []
        return sorted(json.loads(self.transcript_path.read_text(encoding="utf-8"))["segments"], key=lambda
               s: s["start"])

    # ------------------------------------------------------------------ run
    def run(self, force=False) -> None:
        if self.out.exists() and not force:
            if json.loads(self.out.read_text()).get("config_hash") == self.cache_key():
                log.info("cache hit for %s, skipping", self.video_id); return
        random.seed(self.cfg.get("seed", 42)); np.random.seed(self.cfg.get("seed", 42))
        p = self.p5
        sd = json.loads(self.shots_path.read_text())
        fps, n_frames, shots = sd["fps"], sd["n_frames"], sd["shots"]
        starts = [s["start"] for s in shots]
        shot_of = lambda t: max(0, bisect_right(starts, t + 1e-6) - 1)
        cand_idx = [f["idx"] for s in shots for f in s["frames"]]
        idxs = build_detection_indices(fps, n_frames, self._segments(), cand_idx, p["detection"])
        log.info("analysing %d frames", len(idxs))

        # 1. detect + embed (original-frame pixels)
        faces, embs = [], []
        for idx, frame in iter_frames(self.video_path, idxs):
            for d in detect_faces(self.analyzer, frame, p["detection"]):
                d.update(face_idx=len(faces), idx=idx, t=idx / fps, shot_id=shot_of(idx / fps))
                embs.append(d.pop("emb")); faces.append(d)
        E = np.asarray(embs, dtype=np.float32).reshape(-1, 512 if not embs else len(embs[0]))
        log.info("%d faces", len(faces))

        # 2. track inside each shot (a track can never cross a shot boundary)
        by_shot = defaultdict(list)
        for f in faces:
            by_shot[f["shot_id"]].append(f)
        tracks, nid = [], 0
        for sid in sorted(by_shot):
            trs, nid = track_shot(by_shot[sid], E, p["tracking"], nid)
            for tr in trs:
                tr["shot_id"] = sid
            tracks += trs

        # 3. summarise tracks, cluster the reliable ones into characters
        ip = p["identity"]
        for tr in tracks:
            tr["emb"], tr["quality"] = summarise_track(tr, E, ip["top_k"])
            tr["t_start"], tr["t_end"] = tr["faces"][0]["t"], tr["faces"][-1]["t"]
            tr["character_id"], tr["id_conf"] = UNKNOWN, 0.0
        elig = [tr for tr in tracks if tr["quality"] >= ip["min_track_quality"] and len(tr["faces"]) >= ip
              ["min_track_faces"]]
        if elig:
            M = np.stack([tr["emb"] for tr in elig])
            cannot = [(i, j) for i, j in combinations(range(len(elig)), 2)
                      if elig[i]["t_start"] <= elig[j]["t_end"] and elig[j]["t_start"] <= elig[i]["t_end"]
```

```
                        ]
            labels = constrained_average_linkage(M, cannot, ip["cluster_threshold"])
            groups = defaultdict(list)
            for tr, l in zip(elig, labels):
                groups[int(l)].append(tr)
            for members in groups.values():
                c = np.mean([m["emb"] for m in members], axis=0); c /= np.linalg.norm(c) + 1e-12
                for m in members:  # confidence: cosine to cluster centroid; singletons fall back to quali
                      ty
                    m["id_conf"] = float(m["emb"] @ c) if len(members) > 1 else m["quality"]
                    m["_cluster"] = id(members)
            keep = [tr for tr in elig if tr["id_conf"] >= ip["min_id_conf"]]
            order = {}
            for tr in sorted(keep, key=lambda t: t["t_start"]):   # CHAR_01.. by first appearance
                order.setdefault(tr["_cluster"], f"CHAR_{len(order) + 1:02d}")
                tr["character_id"] = order[tr["_cluster"]]

        # 4. write artifacts
        track_char = {tr["track_id"]: tr["character_id"] for tr in tracks}
        for d in (self.faces_path, self.emb_path, self.out):
            d.parent.mkdir(parents=True, exist_ok=True)
        np.save(self.emb_path, E)
        self.faces_path.write_text(json.dumps({
            "video_id": self.video_id, "n_faces": len(faces), "frame_space": "original",
            "faces": [{**{k: v for k, v in f.items()}, "emb_idx": f["face_idx"]} for f in faces]}))
        self.out.write_text(json.dumps({
            "video_id": self.video_id, "config_hash": self.cache_key(), "emb_file": self.emb_path.as_posix
                  (),
            "tracks": [{
                "track_id": tr["track_id"], "shot_id": tr["shot_id"], "character_id": tr["character_id"],
                "id_conf": round(tr["id_conf"], 3), "quality": round(tr["quality"], 3), "n_faces": len(tr[
                      "faces"]),
                "t_start": tr["t_start"], "t_end": tr["t_end"],
                "frames": [{"t": f["t"], "box": f["box"], "mouth": f["mouth"], "emb_idx": f["face_idx"],
                            "det": f["det"], "q": f["q"]} for f in tr["faces"]]} for tr in tracks],
        }, indent=1))
        self.make_track_sheet(tracks)
        self._track_char = track_char

    # ------------------------------------------------------------ evaluate
    def evaluate(self, gt: dict | None = None) -> dict:
        td = json.loads(self.out.read_text()); fd = json.loads(self.faces_path.read_text())
        shots = json.loads(self.shots_path.read_text())["shots"]
        tracks, faces = td["tracks"], fd["faces"]
        n = max(1, len(faces))
        unk_faces = sum(tr["n_faces"] for tr in tracks if tr["character_id"] == UNKNOWN)
        m = {"n_faces": len(faces), "n_tracks": len(tracks),
             "n_characters": len({tr["character_id"] for tr in tracks} - {UNKNOWN}),
             "unknown_rate": unk_faces / n,
             "mean_faces_per_track": float(np.mean([tr["n_faces"] for tr in tracks])) if tracks else 0.0,
             **check_invariants(tracks, shots)}
        if gt:
            tid = {tr["track_id"]: tr["character_id"] for tr in tracks}
            m.update(evaluate_against_gt(gt, faces, tid, self.p5["eval"]))
        return m

    def gate(self, m: dict) -> tuple[bool, str]:
        g, fails = self.p5["gate"], []
        if m["tracks_spanning_shots"]: fails.append("a track spans a shot boundary")
        if m["same_time_same_character"]: fails.append("simultaneous tracks share a character_id")
        r = m.get("recall_medium_large_frontal")
        if r is not None and r < g["medium_large_frontal_recall"]: fails.append(f"recall(medium/large fron
              tal) {r:.2f}")
        if "id_switches" in m and m["id_switches"] > g["max_id_switches"]: fails.append(f"{m['id_switches'
              ]} ID switches")
        if "cluster_purity" in m and m["cluster_purity"] < g["cluster_purity"]: fails.append(f"purity {m['
              cluster_purity']:.2f}")
        return (not fails, "; ".join(fails) or "all gates passed")

    def execute(self, gt=None, force=False) -> bool:
        t0 = time.time()
        self.run(force=force)
        m = self.evaluate(gt)
        ok, why = self.gate(m)
        m.update(runtime_s=time.time() - t0, passed=ok, reason=why, cfg=self.cache_key())
        self.metrics_path.write_text(json.dumps(m, indent=2))
        log.info("metrics: %s", json.dumps(m))
        return ok

    # ------------------------------------------------------ visual inspection
```

```
    def make_track_sheet(self, tracks: list[dict]) -> None:
        """One crop per track (its best face) labelled track / character / confidence: the 'do CHAR_01 and
        CHAR_07 look like the same person?' check from the guide."""
        if not tracks:
            return
        cs, best = self.p5["track_sheet"], {tr["track_id"]: max(tr["faces"], key=lambda f: f["q"]) for tr
              in tracks}
        need = defaultdict(list)
        for tr in tracks:
            need[best[tr["track_id"]]["idx"]].append(tr)
        tiles = {}
        for idx, frame in iter_frames(self.video_path, list(need)):
            for tr in need[idx]:
                x0, y0, x1, y1 = best[tr["track_id"]]["box"]
                mx, my = int(0.2 * (x1 - x0)), int(0.2 * (y1 - y0))
                crop = frame[max(0, y0 - my): y1 + my, max(0, x0 - mx): x1 + mx]
                t = cv2.resize(crop, (cs["thumb"], cs["thumb"]))
                cv2.putText(t, f"{tr['character_id']}", (3, 14), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255,
                      255), 1)
                cv2.putText(t, f"t{tr['track_id']} s{tr['shot_id']} {tr['id_conf']:.2f}", (3, cs["thumb"]
                      - 5),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.35, (255, 255, 0), 1)
                tiles[tr["track_id"]] = t
        arr = [tiles[k] for k in sorted(tiles)]
        if not arr:
            return
        while len(arr) % cs["cols"]:
            arr.append(np.zeros_like(arr[0]))
        sheet = np.vstack([np.hstack(arr[i:i + cs["cols"]]) for i in range(0, len(arr), cs["cols"])])
        self.results_dir.mkdir(parents=True, exist_ok=True)
        cv2.imwrite(str(self.results_dir / f"{self.video_id}_tracks.jpg"), sheet)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--config", default="configs/phase5.yaml")
    ap.add_argument("--video", required=True)
    ap.add_argument("--force", action="store_true")
    ap.add_argument("--gt", help="labelled faces JSON: {'frames': [{'t', 'faces': [{'box','character','fro
          ntal'}]}]}")
    a = ap.parse_args()
    cfg = yaml.safe_load(open(a.config))
    Path("logs").mkdir(exist_ok=True)
    logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s",
                        handlers=[logging.FileHandler(f"logs/{a.video}.log"), logging.StreamHandler()])
    gt = json.load(open(a.gt)) if a.gt else None
    raise SystemExit(0 if Phase5Stage(cfg, a.video).execute(gt, force=a.force) else 1)


if __name__ == "__main__":
    main()
```

16. tests/test_phase5.py

Phase 5 tests with a fake detector on a synthetic video.

```
import json
from pathlib import Path
import cv2
import numpy as np
import pytest
import yaml

from src.evaluation.face_metrics import bcubed, id_switches, match_boxes, pairwise_f1, purity
from src.vision.clustering import constrained_average_linkage
from src.vision.face_detection import face_quality, mouth_centre
from src.vision.phase5 import Phase5Stage, build_detection_indices

FPS, W, H, S = 25, 400, 240, 90
COL = {"A": (0, 0, 255), "B": (0, 255, 0)}


def person_emb(name):
    v = np.random.RandomState({"A": 1, "B": 2}[name]).randn(512)
    return v / np.linalg.norm(v)


class FakeAnalyzer:
    """Finds the solid-colour squares drawn by _make_video and pretends they are faces."""
    def __init__(self): self.rs = np.random.RandomState(0)
```

```
    def analyze(self, bgr):
        out = []
        for name, col in COL.items():
            mask = cv2.inRange(bgr, np.clip(np.array(col) - 5, 0, 255).astype(np.uint8),
                               np.clip(np.array(col) + 5, 0, 255).astype(np.uint8))
            ys, xs = np.where(mask > 0)
            if not len(xs): continue
            x0, x1, y0, y1 = int(xs.min()), int(xs.max()) + 1, int(ys.min()), int(ys.max()) + 1
            w, h = x1 - x0, y1 - y0
            e = person_emb(name) + 0.01 * self.rs.randn(512); e /= np.linalg.norm(e)
            kps = [[x0 + .3 * w, y0 + .35 * h], [x0 + .7 * w, y0 + .35 * h], [x0 + .5 * w, y0 + .55 * h],
                   [x0 + .35 * w, y0 + .75 * h], [x0 + .65 * w, y0 + .75 * h]]
            out.append({"box": [x0, y0, x1, y1], "kps": kps, "det": 0.99, "emb": e.astype(np.float32)})
        return out


def _draw(f, name, x, y): cv2.rectangle(f, (x, y), (x + S, y + S), COL[name], -1)


def _make_video(path):
    vw = cv2.VideoWriter(str(path), cv2.VideoWriter_fourcc(*"mp4v"), FPS, (W, H))
    for i in range(150):
        f = np.zeros((H, W, 3), np.uint8)
        if i < 50: _draw(f, "A", 20 + 2 * i, 50)                    # shot 0: A moves
        elif i < 100: _draw(f, "A", 30, 60); _draw(f, "B", 250, 60)   # shot 1: A and B
        else: _draw(f, "B", 150, 80)                                   # shot 2: B
        vw.write(f)
    vw.release()


@pytest.fixture
def stage(tmp_path):
    cfg = yaml.safe_load(open(Path(__file__).parents[1] / "configs" / "phase5.yaml"))
    cfg["data_dir"], cfg["results_dir"] = str(tmp_path / "data"), str(tmp_path / "results")
    d = tmp_path / "data"
    for sub in ("raw", "shots", "subtitles"): (d / sub).mkdir(parents=True)
    _make_video(d / "raw" / "t.mp4")
    shots = [{"shot_id": k, "start": 2.0 * k, "end": 2.0 * (k + 1), "frames": []} for k in range(3)]
    (d / "shots" / "t.json").write_text(json.dumps({"fps": FPS, "n_frames": 150, "duration_s": 6.0, "confi
          g_hash": "x",
                                                    "crop_box": [0, 0, W, H], "shots": shots, "segment_can
                                                          didates": {}}))
    segs = [{"segment_id": 1, "start": 0.0, "end": 1.9}, {"segment_id": 2, "start": 2.1, "end": 3.9},
            {"segment_id": 3, "start": 4.1, "end": 5.9}]
    (d / "subtitles" / "t.json").write_text(json.dumps({"segments": segs}))
    return Phase5Stage(cfg, "t", analyzer=FakeAnalyzer())


def test_cluster_threshold_and_cannot_link():
    E = np.array([[1, 0], [0.99, 0.14], [0, 1]], float); E /= np.linalg.norm(E, axis=1, keepdims=True)
    assert list(constrained_average_linkage(E, [], 0.3)) == [0, 0, 1]
    assert list(constrained_average_linkage(E, [(0, 1)], 0.3)) == [0, 1, 2]   # simultaneous tracks stay a
          part


def test_quality_and_mouth():
    q_front, fr = face_quality([0, 0, 112, 112], [[30, 40], [80, 40], [55, 60], [40, 85], [70, 85]], 1.0)
    q_prof, fp = face_quality([0, 0, 112, 112], [[30, 40], [80, 40], [100, 60], [40, 85], [70, 85]], 1.0)
    assert fr > 0.99 and fp < 0.2 and q_front > q_prof
    assert mouth_centre([[0, 0]] * 3 + [[40, 80], [60, 90]]) == [50.0, 85.0]


def test_identity_metrics():
    t, p = ["a", "a", "b", "b"], [0, 0, 1, 1]
    assert pairwise_f1(t, p)["pairwise_f1"] == 1.0 and purity(t, p) == 1.0 and bcubed(t, p)["bcubed_f1"] =
          = 1.0
    assert purity(["a", "a", "b", "b"], [0, 0, 0, 0]) == 0.5
    assert match_boxes([[0, 0, 10, 10]], [[1, 1, 11, 11], [50, 50, 60, 60]], 0.5) == [(0, 0)]
    recs = [{"shot_id": 0, "gt": "a", "track_id": k, "t": i} for i, k in enumerate([1, 1, 2, 2])]
    assert id_switches(recs) == (1, 1)


def test_detection_indices_grid_and_union():
    idx = build_detection_indices(25, 150, [{"start": 0.0, "end": 0.5}, {"start": 0.4, "end": 1.0}], [7],
                                  {"det_fps": 5, "window_pad_s": 0.0})
    assert idx == sorted(set(idx)) and 7 in idx and all(i % 5 == 0 for i in idx if i != 7)


def test_pipeline_end_to_end(stage):
    gt = {"frames": [
```

```
    {"t": 5 / FPS, "faces": [{"box": [30, 50, 30 + S, 50 + S], "character": "alice", "frontal": True}]
          },
    {"t": 60 / FPS, "faces": [{"box": [30, 60, 30 + S, 60 + S], "character": "alice", "frontal": True}
          ,
                              {"box": [250, 60, 250 + S, 60 + S], "character": "bob", "frontal": True}
                                    ]},
    {"t": 110 / FPS, "faces": [{"box": [150, 80, 150 + S, 80 + S], "character": "bob", "frontal": True
          }]}]}
assert stage.execute(gt=gt, force=True)
td = json.loads(stage.out.read_text()); m = json.loads(stage.metrics_path.read_text())
tr = td["tracks"]
assert len(tr) == 4 and m["n_characters"] == 2 and m["unknown_rate"] == 0
assert m["tracks_spanning_shots"] == 0 and m["same_time_same_character"] == 0
by_shot = {}
for t in tr: by_shot.setdefault(t["shot_id"], []).append(t)
a0 = by_shot[0][0]["character_id"]; b2 = by_shot[2][0]["character_id"]
assert a0 != b2 and a0 in {t["character_id"] for t in by_shot[1]} and b2 in {t["character_id"] for t i
      n by_shot[1]}
assert a0 == "CHAR_01"                                          # numbered by first appearance
assert m["det_recall"] == 1.0 and m["det_precision"] == 1.0 and m["recall_medium_large_frontal"] == 1.
      0
assert m["cluster_purity"] == 1.0 and m["id_switches"] == 0
for t in tr:                                                    # mouth lies inside its face box
    for f in t["frames"]:
        assert f["box"][0] <= f["mouth"][0] <= f["box"][2] and f["box"][1] <= f["mouth"][1] <= f["box"
              ][3]
E = np.load(stage.emb_path); assert E.shape[1] == 512 and E.shape[0] == json.loads(stage.faces_path.re
      ad_text())["n_faces"]
stage.run(force=True)                                           # deterministic on rerun
assert [(t["track_id"], t["character_id"]) for t in json.loads(stage.out.read_text())["tracks"]] == [(
      t["track_id"], t["character_id"]) for t in tr]
```